
Assignment 1: The CNN Challenge

ITCS 6169/8169 – Computer Vision

Ba-Thinh Lam

Department of Computer Science, University of North Carolina at Charlotte
blam5@charlotte.edu

Code & checkpoint: <https://github.com/sasukepn1999/itcs6169-assignment1> **Test accuracy:** $93.1 \pm 0.8\%$ **Selected-model val. accuracy:** $93.7 \pm 0.5\%$

1 Final Result

Our final model reaches **$93.1 \pm 0.8\%$** accuracy on the test set (seeds: 93.5, 92.0 and 93.8%), compared with 42.3% for the starter network. All numbers are the mean and standard deviation over three seeds.

Validation strategy. We split the 2,400 labelled training images once into 1,920 images for training and 480 for validation, with the same seeded split as the starter notebook. Every design decision is made on this validation set, and the test set is only used for the final models. Since validation accuracy fluctuates from epoch to epoch, the best epoch gives an optimistic estimate, so we compare models by the average accuracy over the last 10 epochs. Our selected configuration reaches **$93.7 \pm 0.5\%$** .

Test evaluation. Our final model also uses FixMatch [3] on the 400 unlabelled images of `test2`. For an honest test accuracy, we train it on the training and validation images only, so it never sees the test images (neither labels nor pixels).

Code and checkpoints. The code is on GitHub and the checkpoints on Google Drive. We provide two versions of the final model with the same recipe: the *clean-test* version, trained on train+val, gives the test accuracy above, while the *submitted* version is also trained on the labelled test images and produces our `test2` predictions (it has no test score, since no labelled image is left).

2 Our Secret Recipe

Our final model is an EfficientNet-B0 [1] initialised from ImageNet-1k [2] weights. We replace the last linear layer with a 16-way classifier and fine-tune all layers. The network takes 200×200 grayscale images, replicated to three channels and normalised with the ImageNet statistics. During training, we apply random-resized cropping, random translation and horizontal flipping. We train for 200 epochs with Adam (batch size 64), with a peak learning rate of 10^{-4} that decays to zero with a cosine schedule, and keep the last-epoch weights. In addition to the cross-entropy loss on labelled images, we apply FixMatch [3] on the unlabelled `test2` images: the prediction on a weakly augmented view becomes a pseudo-label when its confidence exceeds 0.95, and the model learns to predict this label on a strongly augmented view (RandAugment [4] and Cutout [5]).

Which choices mattered? As shown in Table 1, three choices account for most of the improvement: data augmentation combined with longer training, which removes the overfitting of the baseline (+16 points), a deeper network (+15 points) and ImageNet pretraining (+10 points). More labels and FixMatch add a few more points. Finally, we keep every input in grayscale to avoid a colour shortcut (Sec. 4).

3 Experimental Journey

Dataset. The dataset is balanced (150 training and 25 test images per class), so no re-balancing is needed. Two observations shaped our design: images are much larger than the 64×64 input of the starter network, and Flower is the only class stored in colour, so a model could recognise it from colour alone.

ITCS 6169/8169 Computer Vision – Assignment 1 Report.

Table 1: Validation accuracy (%) as each change is added. *Last-10*: average of the last 10 epochs, used for selection.

Experiment	Best	Last-10	Observation
Starter network (64^2 , 20 epochs)	49.6 $\pm$ 0.8	46.9 $\pm$ 1.4	Overfits after $\sim$ 6 epochs
+ Random translation	54.0 $\pm$ 1.6	51.0 $\pm$ 1.2	Rotation, jitter, erasing: no gain
+ Crop and flip, 100 epochs	63.7 $\pm$ 1.5	60.5 $\pm$ 1.1	Still improving
+ Cosine schedule, 200 epochs	66.3 $\pm$ 1.5	65.2 $\pm$ 0.9	Constant lr at 600 epochs: 64.3
+ Input 200×200	69.7 $\pm$ 1.4	68.3 $\pm$ 0.2	256×256 : 61.9
+ EfficientNet-B0 (from scratch)	85.0 $\pm$ 0.2	83.5 $\pm$ 0.5	ResNet-50: 83.5; ConvNeXt-T: 74.5
+ ImageNet pretraining	94.7 $\pm$ 0.7	93.7$\pm$0.5	Larger learning rates: 90.6–92.5

Baseline. The starter network (one convolution layer, 58K parameters) reaches 49.6% validation and 42.3% test accuracy. Its training loss drops to 0.12 while the validation loss rises after six epochs, so we first tackle overfitting.

Augmentation and training length. We add standard augmentations one at a time. Random translation helps the most and almost closes the gap between training and validation loss, while random-resized cropping and flipping help less. In contrast, rotation, colour jitter and random erasing do not help. Since the model is then still improving at the end of training, we combine the three useful augmentations and train longer, until accuracy saturates at about 200 epochs.

Learning-rate schedule. With a constant learning rate, the model still improves slowly after 500 epochs, which suggests decreasing the learning rate during training. A cosine schedule with a larger peak learning rate (5×10^{-3}) reaches higher accuracy in 200 epochs than a constant one in 600.

Resolution. Larger inputs help the starter network, and 200×200 gives the best accuracy. Interestingly, 256×256 was the best size before we fixed overfitting but the worst afterwards.

Architecture. At this point, the training loss of the starter network stays near 1.0, i.e., the network is too small to fit the augmented data. We therefore compare seven standard CNNs trained from scratch. All reach 81–84% except ConvNeXt-Tiny [7] (74.5%), which likely needs more data. We keep EfficientNet-B0, which matches ResNet-50 [6] with $6\times$ fewer parameters.

Self-supervised learning. Since the deeper networks overfit again, we add a self-supervised loss as a regulariser: rotation prediction [8], SimCLR [9], SimSiam [10] or masked image modelling [11]. None of them clearly helps, whether applied to the training or the validation images.

More labels and FixMatch. On the test set, training EfficientNet-B0 (from scratch) on train+val instead of the training split alone improves test accuracy from 81.8% to 84.8%. FixMatch on the unlabelled test images further reaches 86.9%, although its pseudo-label accuracy drops from 95% to 87% late in training (confirmation bias).

ImageNet pretraining. Finally, initialising EfficientNet-B0 from ImageNet weights gives the largest single gain, from 83.5% to 93.7%. A small learning rate (10^{-4}) works best, while larger ones degrade accuracy, probably because they overwrite the pretrained features.

4 Failure Analysis

Keeping the colour of the Flower images. We tried to keep the Flower images in colour, expecting colour to help recognise flowers. Surprisingly, validation accuracy decreased (from 83.5% to 80.9% for EfficientNet-B0), and even the accuracy on Flower did not improve. When we converted the Flower validation images to grayscale at test time, the colour model recognised only 26% of them (vs. 90% in colour), while the grayscale model was unaffected: the model learned the shortcut “colour means Flower” instead of what flowers look like. This shortcut would not hurt our test score, since Flower is also the only colour class in the test set, but it would fail on grayscale flowers. We learned that accuracy alone can hide such a shortcut.

5 AI + Human

AI help: Claude Code help implement all the experiments and draft report. *Human judgement:* I am mainly responsible for designing experiments and hypothesis based on what I observe on the results (e.g., loss curve) and my experience. Details in AI_USAGE.md.

References

- [1] M. Tan and Q. V. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *ICML*, 2019.
- [2] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A large-scale hierarchical image database. In *CVPR*, 2009.
- [3] K. Sohn, D. Berthelot, C.-L. Li, Z. Zhang, N. Carlini, E. D. Cubuk, A. Kurakin, H. Zhang, and C. Raffel. FixMatch: Simplifying semi-supervised learning with consistency and confidence. In *NeurIPS*, 2020.
- [4] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le. RandAugment: Practical automated data augmentation with a reduced search space. In *CVPR Workshops*, 2020.
- [5] T. DeVries and G. W. Taylor. Improved regularization of convolutional neural networks with Cutout. *arXiv:1708.04552*, 2017.
- [6] K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *CVPR*, 2016.
- [7] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie. A ConvNet for the 2020s. In *CVPR*, 2022.
- [8] S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *ICLR*, 2018.
- [9] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *ICML*, 2020.
- [10] X. Chen and K. He. Exploring simple Siamese representation learning. In *CVPR*, 2021.
- [11] K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *CVPR*, 2022.